

**МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ)**

Институт №8 «Компьютерные науки и прикладная математика»

Лабораторная работа №3

по курсу «Технологии параллельного программирования»

Классификация и кластеризация изображений на GPU

Выполнил: Ахметшин Б.Р.

Группа: М8О-403Б-22

Преподаватели: А.Ю. Морозов,

Е.Е. Заяц

Москва, 2026

Условие

Цель работы: научиться использовать GPU для классификации изображений, используя константную память и одномерную сетку потоков.

Выполненная реализация соответствует варианту 3 (метод минимального расстояния): для каждого пикселя выбирается класс с минимальным евклидовым расстоянием до среднего вектора класса в пространстве (r, g, b) .

Программное и аппаратное обеспечение

GPU

Имя	NVIDIA Tesla T4
Compute Capability	7.5
Объем глобальной памяти	15637086208
Разделяемая память на блок	49152
Константная память	65536
Регистров на блок	65536
Макс. потоков на блок	(1024, 1024, 64)
Макс. размер сетки	(2147483647, 65535, 65535)
Количество мультипроцессоров	40

CPU

Архитектура	x86_64
CPU op-mode(s)	32-bit, 64-bit
Address sizes	48 bits physical, 48 bits virtual
Порядок байт	Little Endian
CPU(s)	12
Имя модели	AMD Ryzen 5 7600 6-Core Processor
Ядер / потоков	6 / 12

RAM

64 GB

ОС

Ubuntu 24.04.4 LTS

Компилятор

nvcc, g++

Метод решения

По обучающим пикселям каждого класса вычисляется средний цветовой вектор $avg_j = (\bar{r}, \bar{g}, \bar{b})$. Каждый пиксель изображения классифицируется по правилу:

$$j_c = \arg \min_j ((r - \bar{r}_j)^2 + (g - \bar{g}_j)^2 + (b - \bar{b}_j)^2).$$

Номер класса записывается в альфа-канал выходного изображения.

Описание программы

- `lab3.cu` — GPU-реализация метода минимального расстояния: вычисление средних классов, копирование в `__constant__` память и классификация всех пикселей в одномерной CUDA-сетке.
- `lab3-cpu.cpp` — эквивалентная CPU-версия для сравнения.
- `converter.py` — конвертер `bin <-> png`, включая режим `-alpha-map` для визуализации карты классов из alpha-канала.
- `run_case.sh` — запуск программы по входному изображению и файлу выборок (`cases/case_512.samples`, `cases/case_4096.samples`).

Внутренние замеры:

- GPU: метрики `kernel_ms` и `total_ms`; лог по умолчанию в файле `timings gpu lab3.txt`.
- Размер блока GPU задаётся переменной окружения `LAB3_BLOCK_SIZE`.
- CPU: метрика `total_ms`; лог по умолчанию в файле `timings cpu lab3.txt`.
- Для GPU и CPU имя файла логирования можно переопределить переменной `LAB3_TIMING_FILE`.

Результаты

Тестовые изображения: из ЛР2, размеры 512x512 и 4096x4096.

No	image	CUDA (grid, block)	CPU, ms	GPU kernel, ms	GPU total, ms
1	512x512	<<<8192, 32>>>	0.765451	0.048576	0.850404
2	512x512	<<<2048, 128>>>		0.033280	0.847806
3	512x512	<<<1024, 256>>>		0.036288	0.857774
4	512x512	<<<2048, 512>>>		0.035808	0.835687
5	4096x4096	<<<131072, 128>>>	48.696472	0.724704	30.271024
6	4096x4096	<<<32768, 512>>>		0.749376	29.882846
7	4096x4096	<<<16384, 1024>>>		0.901376	30.117456

Скриншоты:

- входное изображение;
- карта классов (через `converter.py -alpha-map`).



Рис. 1: Входное изображение и карта классов для кейса 512x512

Выводы

По полученным замерам видно, что время ядра GPU мало и составляет доли миллисекунды: для 512×512 примерно 0.033–0.049 мс, для 4096×4096 примерно 0.725–0.901 мс. При этом совокупное время GPU заметно больше времени ядра, так как включает накладные расходы на обмен данными и служебные операции.

На размере 512×512 CPU и GPU дают близкие результаты по полному времени: CPU 0.765 мс, GPU 0.836–0.858 мс. На размере 4096×4096 GPU уже быстрее CPU: CPU 48.696 мс против GPU 29.883–30.271 мс (ускорение около 1.6 *times*).

По совокупному времени лучшая конфигурация блока в замерах: <<<2048, 512>>> для 512×512 и <<<32768, 512>>> для 4096×4096 . Карта классов для изображения 512×512 показывает корректное разделение на два класса, что подтверждает работоспособность реализации.